

R Lab 1: Introduction to R and R Notebooks

Learning objectives

In this lab, you will learn how to do the following:

1. How to set up an R Notebook and how to organize R Notebooks on your computer
2. How to install packages
3. How to enter R code in **chunks**
4. How to enter notes to yourself in **annotations**
5. How to do basic R commands
6. How to create objects to hold lists of numbers using the `c()` command

Introduction

In this class, we will be using R, along with R Studio to analyze data. RStudio is an IDE (interactive development environment) that runs with R and makes it easier to use. Before you start this lab, you should install R and RStudio on your computer and ensure that it is running.

Why use R?

Why should I spend time learning how to use R and RStudio? There are several reasons and strengths to R, though it can be hard to use at times.

1. R and RStudio is free and open source. That means it costs nothing, ever. Many software packages are free for students and then cost when you're outside of school. Others are not free. Would you like to pay \$100 or more for a software package?
2. R is a standard in statistics analysis. Along with Python (which is even harder), it's the standard that people use in the profession. May as well learn how to use the right tools now.
3. R and R notebooks are reproducible. One of the biggest issues in data analysis, across any science, is reproducibility. It is one thing if a researcher has trouble reproducing another researcher's results. But what if a person can't reproduce their own results?

In grad school, I would do analyses and get results, and then have no idea how I could get those same results again. Or I had no way to know how I got from my raw data to the finalized data after cleaning and editing it. I thought I was following the same steps, and doing the same things, but I didn't know what I was doing.

4. R shows all your analysis steps. When I analyzed data before I used R, I could have been making mistakes and not knowing it. I made several mistakes that I caught, but what if there were others? I took good notes, but what if I clicked the wrong button and didn't know it. One of the strengths of R is that you can create a text-based log of all your analyses through R Notebooks. RStudio makes this easy, allowing you to combine your R commands and your notes for your commands into a single document that you can export.

This assignment

For this assignment, I want you to create a basic R notebook, place some code in chunks and some notes in annotations, and then to knit it.

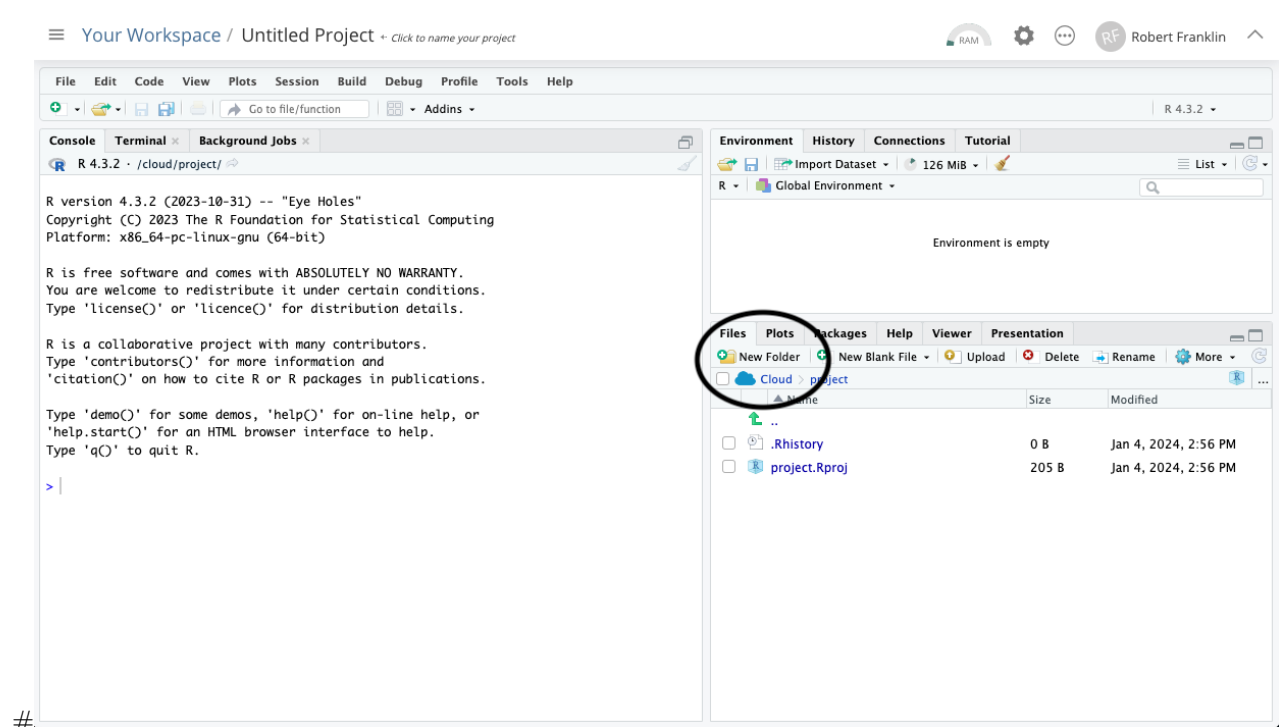
Organizing Assignments and Creating an R notebook

The way I suggest you organize your assignments is to create a folder for this class on your computer, and then create a subfolder inside that folder that holds each assignment. R notebooks create additional files when they are knitted and the folder for each assignment will hold these folders in an organized place. In addition, you can place any data files required for each assignment in that folder.

If you are using Posit Cloud, you will want to create a folder on the Posit Cloud server. You will use this folder to upload any data.

Step 1: Create a folder for this class if you have not already done so. If you don't normally create folders, do this on your desktop by right-clicking or clicking with two fingers (Mac) and make a folder for this class. You can call it "Stats" or whatever you want

If you are using Posit cloud, first open RStudio. Create a new folder by clicking on the New Folder in the top of the window on the bottom half of the screen. You might have to click the "Files" tab to see this:



Step 2: In the folder for this class, create a folder in that folder that is named 'Lab 1'.

Now we will create an R notebook.

Step 3: Go ahead and open R Studio.

Step 4: Open a new notebook file by clicking File -> New File -> New Notebook. The first time you open an R notebook, RStudio may ask you to install several packages. Go ahead and accept this and wait until it is finished.

R Notebooks have two parts. The first is called the *front matter* and those are some options which are at the beginning. This should be the default title and an output. The front matter is separated by three dashes '—'. Don't delete these three dashes, or else bad things will happen.

Step 5: Change the title to “Lab 1”. Make sure to keep the quotes, so R knows this is a title.

The rest of the R Notebook is below the front matter. The basic R notebook gives you a template with some information, but we don’t need this.

Step 6: Delete everything below the second set of three dashes. Make sure to keep the three dashes. This should be deleting everything below line 5.

Step 7: Now, save this new notebook on your computer in the folder you created for this assignment by clicking File -> Save As.

About R Notebooks

The main part of R notebooks are broken into two sets. There are annotations, which are words you type into the notebook. This is just like typing text.

Installing and loading packages

Packages are add-ons to R that allow it to do so much more than the basic things. The packages is what makes R so powerful because it can allow for people to add new analyses and programs to do any type of new analysis.

Packages are add-ons that have to be installed *once* and then **loaded** every time you make an R notebook. I will tell you which packages you need to install and load. If you ever get an error like this:

Then you know that you have not installed the package and you should do so.

The first packages we will install is called **tidyverse**. It’s a package that has a lot of other packages we will use for graphing and formatting data.

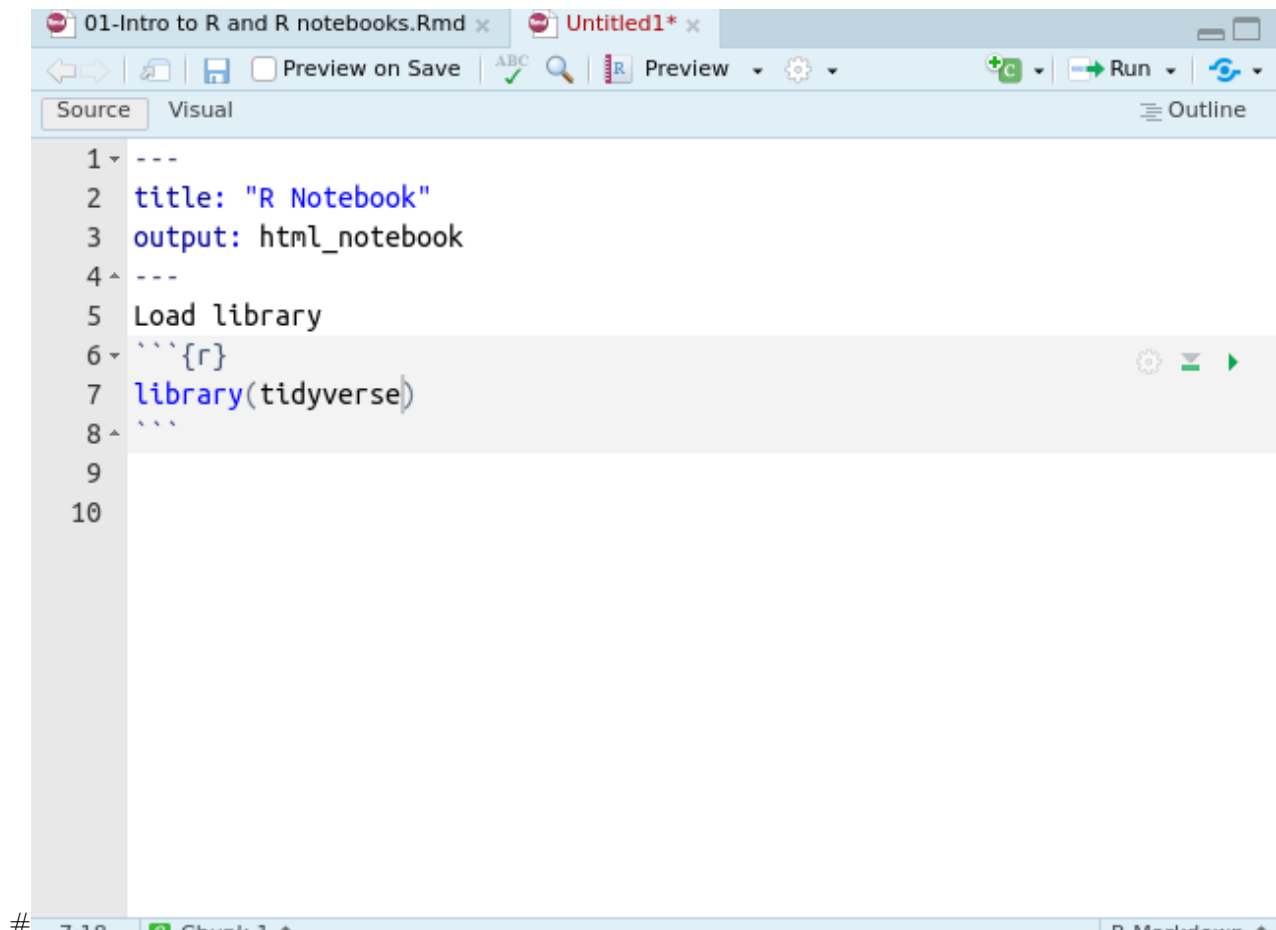
Step 8: Install the tidyverse package by clicking the Packages tab in the bottom-left window. You will see a tab that says something like “Files”, “Plots”, “Packages”, “Help”, “Viewer”. Click the “Packages” tab and then click the Install button.

Step 9: In the box that pops up, type ‘tidyverse’ and hit enter. You will notice that it will give you a suggested hint once you type part of the word. You can select it after typing part of it. There will be a lot of messages that appear in your console. If there are any messages that ask you to make any choices, just follow the prompts and accept them. Sometimes, it might stop and ask you to type Y to continue. If so, go ahead and do so. You only have to install a package once. However you will have to load the packages you need each time you do an R notebook.

Step 10: Now we will create our first annotation. In the first line below the three dashes ---, type “Loading packages” (no quotes needed, just type it straight into the R notebook).

Step 11: Go to the next line to create a code chunk. Make a code chunk by going to Code -> Insert Chunk (or pressing “Control-Alt-I” (Windows) or “Command-Option-I” on Mac). In that code chunk, type `library(tidyverse)` just like indicated below.

```
library(tidyverse)
```



Once you type the code chunk, you can run the chunk by pushing the tiny green play arrow in the top-right corner of the chunk. This will give some messages. If you run a code chunk, you will either get an output, an error, or nothing.

R Commands in Chunks

Now we should go over the ideas of R commands. In a programming language, a **command** is a statement that does something (Wikipedia says it is a “directive to a computer program to perform a certain task”). In R, commands are what you type into R.

In a programming language, you give a command as an **input** and then receive something as **output**.

Now we will go over doing some basic R commands. When I give you the R commands that are highlighted below, please place them in the chunks and run them. Answer the questions by typing annotations in between the code chunks. You should always annotate your code so that you know what each chunk is doing.

Type the following command as a code chunk. What do you get when you run it?

```
1
```

```
3+4
```

Commands usually give outputs.

When you see output, you will often see a number in brackets, like if you type the output below:

```
3*5
```

```
## [1] 15
```

The number in brackets tells us how many **items** we have in the output. In a simple math problem, there is only one output, the answer. However, we might have many outputs, like a whole list of numbers. The number in brackets helps us if we have a long list of numbers, like if I type the following which lists the numbers 1 through 100.

```
1:100
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100
```

The number in brackets tells me which value in the list is to the right of the number in brackets. So when I see, [18], I know that the number to the right is the 18th item in the list.

Doing basic math in R

Some of the most basic commands are simple math using numbers and math symbols. To add, use the plus key. To subtract, use the minus key. To multiply, use the asterisk key, and to divide, use the forward slash key. Exponents are used by the caret key. See below:

```
3+7
4-11
4*21
18/3
2^2
```

Go ahead and type code chunks to do the following:

9 multiplied by 14

32 divided by 3

11 minus 4

Finally, R follows the basic order of operations, with information in parentheses done first, then exponents, then multiplication and division, followed last by addition and subtraction. So the following are different. Try typing each of these as code chunks and seeing the output.

```
3 + 11*5
```

```
## [1] 58
```

```
(3+11)*5
```

```
## [1] 70
```

Note that R **does not** do multiplication by putting numbers by parentheses. This won't work `3(4+4)` Your algebra teacher would be mad.

Objects

If this is all that R could do, you would wonder why bother. The power of R though is that you can do a lot more than that. You can use things called **objects** which are like containers for data and **functions** which can be lots of commands put together to do very complex things.

An object is like a box. It holds things. It can hold anything from a single number to thousands and thousands of rows of data. Objects have different types, which determines what kind of data it holds.

The simplest type of object is a variable. It is simply a letter or word that stands for a number. To make a variable you use the following (replacing the bracketed number with an actual number (like 42)).

To assign a value to an object, we simply use equals, with the object on the left and the value to assign on the right.

```
x = 1
```

When we assign a value to an object, we don't get an output, since there is nothing for R to say. To see what an object contains, you can type the object's name as a code chunk and run it or look at the environment in the top-right window of the RStudio console.

```
x
```

```
## [1] 1
```

The name of objects can be any text that starts with letters. This can contain letters with numbers or dots in the middle. Objects should not contain spaces. This can be done but is very complex.

Objects also can hold different types of data. For this class, there are types of data we'll discuss. Numeric data are anything that's a number. Character strings are text data and are assigned by using quotes, either single or double.

```
number = 4
name = "Michael"
day.of.week = "Tuesday"
question20 = 5
```

We can also do math on objects, just like numbers. Go ahead and type the following.

```
x = 3
y = 7
x*y
```

```
## [1] 21
```

An important thing to note is that objects will get replaced if you reuse them. If I type the following, I am only using one object, and just replacing the value stored there:

```
t = 3  
t = 5  
t = 11
```

Do the following in R.

1. Create an object that is labeled as your first name and have that object equal your age. Write down what you would type.
2. Make a second object that is the same as the name of the person sitting beside you, and have it equal to their age. If you're not doing this assignment with another person, just pick another person you know and use their name and age.
3. Add the two objects together. What would you type?

Lists and functions

It's pretty useless to do stats on single numbers. We have to put lots of values together to get real stats. To do this, we can create objects called **lists**. To make a list, we use functions.

A function is a command that takes inputs, does something, and produces outputs. All functions in R have the following format.

```
name(inputs)
```

Each input is separated by a comma:

```
name(input1,input2,input3)
```

Some functions have no inputs at all. In this case, they just have two parentheses with nothing in them.

```
function()
```

The first function we will use is the concatenate function, or `c()`. It joins all the **arguments** into one object, called a list. This function has two or more arguments and has one output, which is a list. Here are a few examples. Type the following.

```
c(1,2,3,4,5)
```

When a function runs, it usually has an output. That output is by default output to the screen below the code chunk, like in your last example. However, the output can also be saved as an object, just like how we made an object above.

Type the following examples.

```
x = c(1,3,5,7)
ages = c(19,22,37,41,18)
IQ = c(97, 132, 88, 101, 94, 107)
presidents = c("George Washington", "John Adams", "Thomas Jefferson")
```

Once you type these, notice these objects are stored in the Environment tab of RStudio on the top right. You can click on these to view the objects.

Lists can be used as the input for functions which need more than one number, such as taking the sum or mean, and so forth. Try the functions below, which give the mean, the sum, and the standard deviation of a few of the lists you just made above:

```
mean(IQ)
```

```
## [1] 103.1667
```

```
sum(x)
```

```
## [1] 16
```

```
sd(ages)
```

```
## [1] 10.78425
```

4. Create an object named `scores` with the values 23, 47, 39, 36, 41, 32, and 41. Calculate the mean and standard deviation of `scores`.

Creating data frames

Variables are nice, but sometimes we want to hold a lot of organized data together, like a table. In R, these are called dataframes, and they have variables as separate columns with each row being an observation.

Person	Name	Age	Worth
1	Alice	45	\$5,000,000
2	Bob	39	\$432,000
3	Cara	29	\$87,000
4	David	66	\$11,000
5	Ethan	54	\$33,000

Each row refers to a set of observations, which is a single person. Each column is a different statistic.

Now we are going to make a dataframe. For example, I may want to make a dataframe with how many hours they slept and their number of items they got correct two different 25 question memory tasks.

Name	Sex	Sleep	Memory Test 1	Memory Test 2
Zoe	Female	9	22	19
Yannick	Male	10	17	20
Xavier	Male	6	15	23

Name	Sex	Sleep	Memory Test 1	Memory Test 2
Wendy	Female	5	13	18
Vance	Male	5	10	8

To do this, I have to make a list for each column and then combine them using the `data.frame()` command into the dataframe called “results”. To do this, type the following (note I keep all the variable names in lower case; this is my convention but make sure you are consistent with capitalization):

```
name = c("Zoe", "Yannick", "Xavier", "Wendy", "Vance")
sex = c("Female", "Male", "Male", "Female", "Male")
sleep = c(9, 10, 6, 5, 5)
memory1 = c(22, 17, 15, 13, 10)
memory2 = c(19, 20, 23, 18, 8)
results = data.frame(name, sex, sleep, memory1, memory2)
```

If you get an error here, it might be because you didn’t type each of the lists correctly. For example, if you miss a value, then one of the lists will have 4 values, instead of 5. Then when you put the values into a dataframe, R won’t know what to do.

To see the dataframe, I can click on the data frame in the Environment tab. Click on the little icon to the right of where it says `results` that looks like a table. This will bring up another window that shows the data.

Additionally, another way to view a dataframe is to just create a chunk that contains the dataframe itself. If you type the following as a chunk, this will output the data. However, I do not often do this because some dataframes can be very big and I don’t want to output the data when I knit the notebook.

```
results
```

If we want to look at a single column or variable in a data frame, we would select it by first listing the data frame name, then a dollar sign, then the column name. To select the column for names, I would type:

```
results$name
```

```
## [1] "Zoe"      "Yannick" "Xavier"  "Wendy"   "Vance"
```

This outputs a column of the data frame as a list.

We can do math to each of the columns. For instance, if I want to create a new column which is the sum of each of the three tests, I would type:

```
results$memory.total = results$memory1 + results$memory2
```

Now I have created a new column called `memory.total`. Notice that I used a period in the middle of the column name to separate the words. This is something you can do to avoid using spaces. However, the column name shouldn’t start or end with a period.

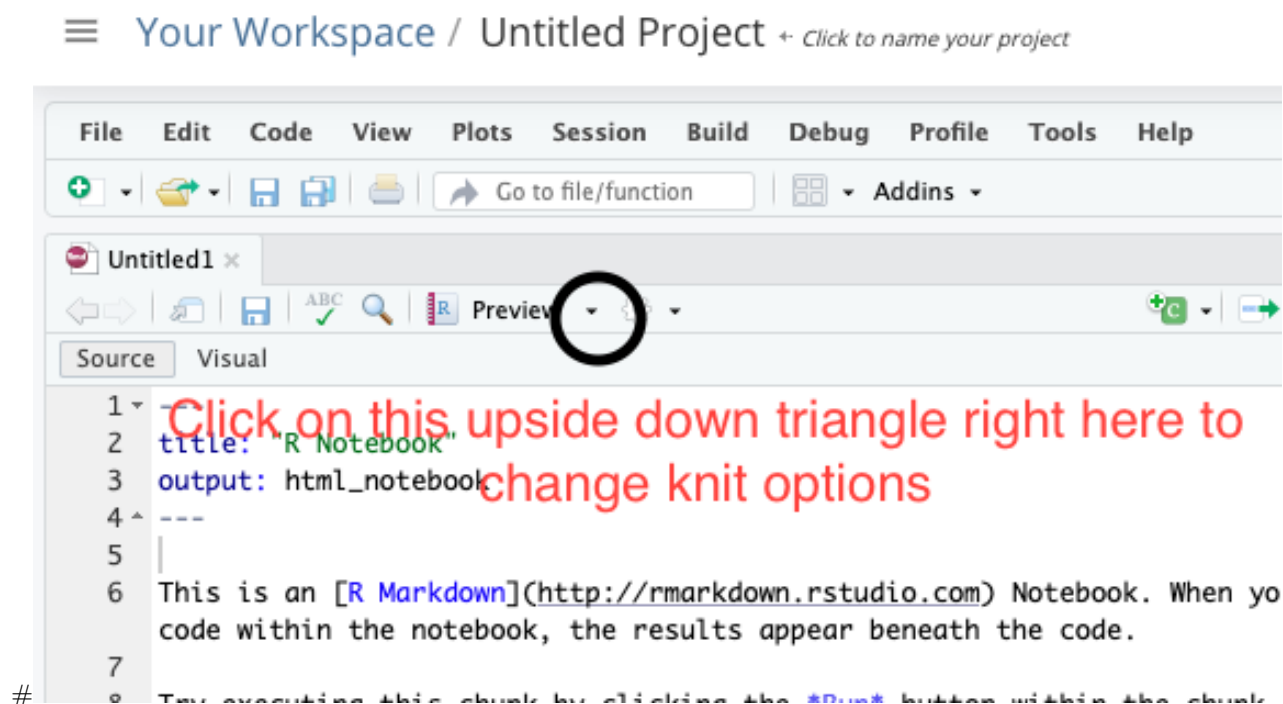
5. What would you type to display the `memory.total` column as a list on the screen? Add this as a chunk and make an annotation to answer the question.

6. How would you create a new column in the data frame called `memory.mean`, which is the mean memory score for the two memory tests. Remember, you could just create this column by dividing the `memory.total` column by 2.

Knitting the R Notebook

Now you're done with Lab 1. It's time to knit the notebook. Knitting the notebook produces a nice output of your data, including graphs and other analyses. In fact, this lab that you're reading is a knitted R notebook.

When you knit your R notebook, you can do so by clicking File -> Knit Document or choosing the tiny drop-down menu above this menu where it says "Preview Notebook" or "Knit Document". From this menu, you can choose what format you want to use to knit the notebook. I like this option better since it allows you to knit it to a word document.



Go ahead and knit the notebook. You will see some text appear in your console. If you see an error, look at the line number where the error is. Then go to that line number and see what is causing the error. There is probably a typo in your code, or you accidentally deleted some of the punctuation around a code chunk. Sometimes you will have part of a code chunk left or put an annotation in a code chunk. Once you fix the error, try again.

Posit Cloud note - To upload your assignment from Posit Cloud to canvas can be difficult, especially if you are using an ipad. After you knit your document, if you knit it as a word document, clicking on it should give you the option to download it. You can then take that downloaded document and upload it to Canvas.

Now you have finished, upload your assignment. If you can't get your document to knit, please let me know, or upload your `.Rmd` file, which is the text of your saved notebook. Then I can see why it's not knitting.

Check your learning

After you do this lab, you should know or be able to do the following:

1. Know what an R notebook is, and know how to create one and save it on your computer
2. Know what the front matter, annotations, and code chunks are in an R notebook and how to use each.
3. Know what commands, objects, variables, lists, and functions are and how to use each
4. Be able to create a list using the `c()` command and calculate the mean, sum, and standard deviation of that list
5. Be able to create a data frame using the `data.frame` command, know how to create new columns based on what is in the data frame, and find the mean and standard deviation of a column.

If you need to do this in a future lab, use this lab as a reference.